VIDYALANKAR INSTITUTE OF TECHNOLOGY

Smart Irrigation Alert Dashboard

Problem Definition, Proposed Solution and Technical Evaluation

Chetan Katkar | Roll Number 23102A0024 | BE CMPN-A

DevOps Engineering | Academic Year 2026-27 | Odd Semester

Prepared 29 October 2026

Executive summary

This project addresses a practical irrigation-monitoring problem: farmers and agricultural managers may not have a centralized view of soil moisture across zones, and fixed schedules or manual checks can delay watering decisions or waste water. The proposed prototype simulates moisture and temperature readings, shows zone status in a browser dashboard, highlights moisture below 30%, and lets an operator simulate an irrigation response.

The current repository implements a Spring Boot and Java 21 web application with a static frontend, REST APIs, Spring Data JPA, and embedded in-memory H2 storage. The DevOps coursework defines a GitHub-Jenkins-Maven-test-Docker-Ansible workflow. The repository contains code and documentation for these stages, but this review did not verify an end-to-end container deployment. Several source documents describe the pipeline and test outcomes more strongly than the implementation and saved execution evidence support.

1. Background and problem statement

Irrigation decisions depend on changing soil conditions across separate crop areas. Manual soil inspection takes time and may not catch a threshold breach promptly. Timer-based irrigation can continue when moisture is already adequate, while a dry area may remain unattended until someone notices the problem. The lack of a shared multi-zone view also makes it harder for a manager or agronomist to compare conditions and prioritize action.

Problem statement: small and medium-scale agricultural operations need a simple, centralized way to view moisture and temperature readings across zones, identify areas below a safe moisture threshold, and take timely action. The academic MVP demonstrates that workflow with simulated sensor data and a local web application; it does not connect to physical field sensors.

2. Users, stakeholders and pain points

Group	Need or interest
Farmers and farm owners	See field conditions, receive critical alerts, and initiate irrigation actions.
Farm managers and agronomists	Compare zones, search conditions, and prioritize low-moisture areas.
System administrators	Build, deploy, and maintain the application and its runtime.
Sensor providers and maintenance teams	Future integration with sensor data formats and field equipment.
Water-conservation bodies and project supervisor	Resource-use visibility, documented scope, and a demonstrable DevOps workflow.

- Manual inspections are time-consuming and depend on people visiting each field.
- Fixed irrigation schedules do not account for current soil moisture.
- Multi-zone readings are not gathered in one shared view.
- Delayed detection can let low moisture persist until a person checks the field.
- Unstandardized deployment steps can make software releases inconsistent.

3. Project objectives and scope

- Provide a dashboard for multiple named zones with moisture, temperature, status, and update time.
- Classify moisture below 30% as critical and expose affected zones through an alert view.
- Allow adding zones and filtering the displayed set by name/crop and status.
- Expose summary values for total zones, average moisture, critical alerts, and optimal zones.

- Support reproducible local build and packaging with Java 21, Maven, Spring Boot, and embedded H2.

The MVP includes simulated data entry, a searchable dashboard, summary indicators, current zone drill-down, and a visual exception view. Physical sensors, a mobile app, SMS/email, machine-learning recommendations, weather integration, and multi-farm tenancy are outside scope. The documented UI drill-down currently shows the latest reading and does not keep the historical series listed in the original user story.

4. Success criteria and constraints

The initial Problem_Statement.pdf sets targets of at least three simulated zones, alert response within five seconds, 100% uptime during testing, commit-to-container deployment under five minutes, and at least three passing Selenium tests. Main_Doc.pdf records a stricter target set of at least five zones, alert visibility below two seconds, 100% green Selenium tests, package execution below three minutes, and one-command container execution. These are conflicting target versions and should be reconciled before being reported as acceptance criteria.

Target area	Evidence available now	Result
Multi-zone display	Five demo zones are seeded in source; saved browser capture shows five.	Implemented in source; current runtime not rechecked.
Threshold alert	API and UI classify moisture below 30% as critical.	Implemented in source.
Response time / uptime	No timing or uptime test report is stored.	Not measured.
Selenium tests	Selenium dependency exists; the tracked suite is five MockMvc tests.	Browser automation not implemented.
Commit-to-container release	Dockerfile and Jenkinsfile exist, but Jenkins has no Docker deployment stages.	End-to-end target not demonstrated.

The prototype assumes a Java 21 runtime, Maven dependencies available from a repository, and a working Docker engine for container builds. Sensor values are simulated. H2 is configured in memory, which removes the external database setup but does not preserve readings across restarts.

5. System design

The browser loads a static HTML interface and calls JSON endpoints. SensorApiController handles REST routes and calls SensorRepository directly. SensorZone is stored through Spring Data JPA in H2. The project architecture PDF includes a separate service and alert-engine layer; the current source has no separate service class, so the implementation is simpler than the proposed diagram.

Component	Implementation role
Browser UI	HTML, CSS, and JavaScript; displays readings and summary data and submits user actions.
REST API	Spring Boot controller routes for list, add, alert, summary, simulation, and irrigation.
Domain model / persistence	SensorZone entity and SensorRepository backed by embedded H2.
Build / runtime	Maven project; Java 21; application port 8085.
DevOps configuration	Jenkinsfile, Dockerfile, and Ansible files are present; execution evidence is incomplete.

Moisture status logic

The application maps readings below 30% to CRITICAL, from 30% through 60% to OPTIMAL, and above 60% to SATURATED. On startup, five named agricultural demo zones are inserted when the database is empty. The simulate endpoint changes readings by bounded random increments. The irrigation endpoint assigns a simulated 75% moisture value.

API summary

Method	Endpoint	Function
GET	/api/sensors	Return current zone readings.
POST	/api/sensors	Create a sensor zone and calculate status.
GET	/api/alerts	Return zones below 30% moisture.
GET	/api/summary	Return count, average, critical, and optimal metrics.
POST	/api/simulate	Change readings to simulate sensor movement.
POST	/api/sensors/{id}/irrigate	Set a zone to 75% moisture.

6. Implementation and DevOps workflow

The repository history records an initial project setup, a sensor-dashboard feature commit and pull-request merge, alert-related changes, a conflict resolution, a v1.0 tag, and a later Linux/Dockerfile adjustment. The local main branch has one commit ahead of origin/main. The tracked Jenkinsfile uses checkout, Maven compile, test, JAR packaging, JUnit report publishing, and artifact archiving. Its deployment stage currently prints text rather than deploying the artifact.

The Dockerfile uses a multi-stage Maven build and a Java 21 JRE runtime, runs the app as a non-root user, and exposes container port 8085. The command guide maps host port 8086 to that container port. The Ansible files document localhost provisioning; however, setup.yml stops and removes the container and launches it again each time, so it does not satisfy the documented second-run changed=0 idempotency claim.

7. Test and runtime evaluation

The project contains five MockMvc integration tests covering sensor retrieval, critical-alert detection, zone creation, summary metrics, and irrigation. Although Selenium is listed as a dependency and is required by the assignment checklist, no Selenium browser test class or screenshot-on-failure mechanism is present. No current Surefire report was available in the repository.

A fresh mvn test attempt in this review could not proceed: the default local Maven repository was read-only in the task execution environment. Retrying with a temporary repository failed because repo.maven.apache.org could not be resolved. Docker context inspection showed desktop-linux, but Docker API access was denied; local HTTP calls were also blocked by the task sandbox. Therefore no new test pass, application response, Docker build, container state, or Jenkins build is claimed here.

8. Current gaps

- Update the README clone placeholder and direct-run port. The app uses 8085, Jenkins defaults to 8080, and the Docker guide uses host 8086.
- Push the local Linux/Dockerfile commit to GitHub after setting up SSH or token-based authentication.
- Add Selenium journeys and actual test reports. Do not describe MockMvc tests as Selenium evidence.

- Change Jenkins deployment from an echo-only stage to a concrete deployment or rename it as a package-only CI stage.
- Add Docker build, registry, deployment, credential, and health-check stages only after the Jenkins agent can access Docker and credentials are configured.
- Make Ansible converge on desired state; add package, user, file, and configuration tasks; then validate two actual runs.
- Implement persistent historical readings, API-level input validation, and durable storage if the application is extended beyond the demo.

9. Future enhancements

- Integrate physical soil-moisture and temperature sensors through a validated ingestion API.
- Persist readings with timestamps and show trend charts, minimum, maximum, and average moisture per zone.
- Add secure users and role-based access for operators and administrators.
- Add event notifications and connect recommendations to valve or pump controls with safe authorization.
- Add weather-aware scheduling, predictive analysis, and multi-farm tenancy after the monitoring foundation is validated.
- Measure response latency, deployment time, availability, and recovery behavior in repeatable test runs.

Conclusion

The Smart Irrigation Alert Dashboard is a useful academic prototype of a sensor-monitoring workflow. Its core UI and REST paths cover zone listing, basic creation, alert detection, summary calculations, simulated reading changes, and irrigation simulation. The project also contains Git history, a Java 21/Maven build definition, a Dockerfile, Jenkins pipeline code, and Ansible configuration. The operational workflow is not yet proven end to end: Selenium tests are absent, Jenkins deployment is a placeholder, Docker is not wired into the current pipeline, and the Ansible playbook contradicts its idempotency claim. Completing these items and saving current Ubuntu logs will make the demonstration defensible.

Project evidence referenced

The following source artifacts were reviewed while preparing this document:

- User-provided CMPN-A.xlsx, sheet BE CMPN A, row for 23102A0024.
- Task-01/Problem_Statement.pdf, Scope.pdf, StakeHolders.pdf; Task-02 planning PDFs; Task-03 SRS, architecture, API, and setup PDFs.
- Current repository source: pom.xml, src/main/java/com/irrigation, src/main/resources, src/test/java, Jenkinsfile, Dockerfile, and Task-13 Ansible files.
- Current repository evidence: Git history/refs, Task-04 to Task-07 screenshots, Task-08 to Task-15 Markdown documentation.